

The Hooks Cheat Sheet

A guard that stops it touching what you cannot afford to lose.

THE GUARD ITSELF

.claude/hooks/protect-files.sh

```
#!/bin/bash
INPUT=$(cat)
FILE_PATH=$(echo "$INPUT" | jq -r '.tool_input.file_path
// empty')
FILE_PATH="${FILE_PATH//\\//}"

PROTECTED=(".env" "package-lock.json" ".git/")

for p in "${PROTECTED[@]}; do
  if [[ "$FILE_PATH" = *"$p"* ]]; then
    echo "Blocked: $FILE_PATH matches '$p'" >&2
    exit 2
  fi
done
exit 0
```

The last four lines are the whole idea. On the list: say why, and `exit 2`. Not on the list: `exit 0` and get out of the way.

SWITCH IT ON

.claude/settings.json

```
{
  "hooks": {
    "PreToolUse": [
      {
        "matcher": "Edit|Write",
        "hooks": [
          { "type": "command",
            "command":
"$\${CLAUDE_PROJECT_DIR}/.claude/hooks/protect-files.sh"
          }
        ]
      }
    ]
  }
}
```

`matcher` is the filter. `Edit|Write` means it runs on edits and writes only. Not reads, not shell commands.

THE ONLY TWO NUMBERS THAT MATTER

NUMBER	MEANING	WHAT YOU NEED TO KNOW
0	No objection	This is not the same as "approved". Claude's normal permission rules still run afterwards.
2	Blocked	The action is stopped. Whatever your script printed as an error goes back to Claude, so it can try something else.

GUARDS WORTH HAVING

GUARD	RUNS	DOES
Protect files	Before an edit	Refuses <code>.env</code> , lockfiles, <code>.git/</code>
Log changes	After an edit	Writes every changed file to a log
Tidy up	After an edit	Runs your formatter
Tell me	When it is waiting	Pings you, so you stop watching

THREE REASONS A GUARD NEVER FIRES

The script is not allowed to run

On macOS and Linux, `chmod +x` it. On Windows there is no such permission, but the script is Bash, so you need Git for Windows. Check with `git --version`.

The path in settings is not where you saved it

The most common one. `/hooks` shows what is registered, read only. If `PreToolUse` shows `0`, the path is wrong.

The matcher is narrower than you think

`Edit|Write` does not cover a file changed by a shell command. That change walks straight past your guard.

THE ONLY TEST THAT COUNTS

Ask it to edit a protected file on purpose. If it edits instead of refusing, the guard is not wired up. Testing the happy path proves nothing.